

Gemini

```
import numpy as np
import matplotlib.pyplot as plt
import math

def solucao_trelica():
    print("--- SISTEMA DE ANÁLISE DE TRELIÇAS ISOSTÁTICAS ---")

    # 1. ENTRADAS DINÂMICAS [cite: 9, 10, 11, 12, 13, 16]
    try:
        N = int(input("Número de nós (N): "))
        coords_x = np.zeros(N)
        coords_y = np.zeros(N)
        for i in range(N):
            coords_x[i] = float(input(f"  Nó {i} - Coord X: "))
            coords_y[i] = float(input(f"  Nó {i} - Coord Y: "))

        B = int(input("Número de barras (B): "))
        barra = np.zeros((B, 2), dtype=int) # Vetor de conectividade [cite: 13]
        for i in range(B):
            barra[i, 0] = int(input(f"  Barra {i} - Nó Origem: "))
            barra[i, 1] = int(input(f"  Barra {i} - Nó Destino: "))

        # Vetores de cargas e apoios [cite: 15, 16]
        Fx = np.zeros(N)
        Fy = np.zeros(N)
        apoio_tipo = [] # 0: Livre, 1: Móvel(Y fixo), 2: Móvel(X fixo), 3: Fixo(XY)

        for i in range(N):
            Fx[i] = float(input(f"  Nó {i} - Carga Fx (N): "))
            Fy[i] = float(input(f"  Nó {i} - Carga Fy (N): "))
            apoio_tipo.append(int(input(f"  Nó {i} - Apoio (0:Livre, 1:Móvel-X, 2:Móvel-Y, 3:Fixo): ")))

    except ValueError:
        print("Erro: Entrada inválida. Use apenas números.")
        return

    # 2. PROCESSAMENTO: MONTAGEM DA MATRIZ
    # Determinar número de reações para compor o sistema 2N x 2N
    num_reacoes = 0
    for t in apoio_tipo:
        if t == 1 or t == 2: num_reacoes += 1
        elif t == 3: num_reacoes += 2

    if (B + num_reacoes) != (2 * N):
        print(f"Aviso: A estrutura pode não ser isostática! (Incógnitas: {B+num_reacoes}, Equações: {2*N})")
```

```
M = np.zeros((2 * N, 2 * N))
R = np.zeros(2 * N)
```

```
# Preencher forças das barras
```

```
for i in range(B):
    n1, n2 = barra[i]
    dx = coords_x[n2] - coords_x[n1]
    dy = coords_y[n2] - coords_y[n1]
    L = math.sqrt(dx**2 + dy**2)
    cos, sen = dx/L, dy/L
```

```
# Componentes nos nós
```

```
M[2*n1, i], M[2*n1+1, i] = cos, sen
M[2*n2, i], M[2*n2+1, i] = -cos, -sen
```

```
# Preencher reações e vetor de cargas
```

```
col_reac = B
for i in range(N):
    R[2*i], R[2*i+1] = -Fx[i], -Fy[i]
    if apoio_tipo[i] == 1: # Reação em Y
        M[2*i+1, col_reac] = 1; col_reac += 1
    elif apoio_tipo[i] == 2: # Reação em X
        M[2*i, col_reac] = 1; col_reac += 1
    elif apoio_tipo[i] == 3: # Reação em X e Y
        M[2*i, col_reac] = 1
        M[2*i+1, col_reac+1] = 1; col_reac += 2
```

```
# 3. IMPLEMENTAÇÃO MANUAL DE GAUSS [cite: 19, 20, 43]
```

```
system = np.column_stack((M, R))
n_total = len(system)
for i in range(n_total):
    # Pivoteamento parcial para robustez
    max_el = abs(system[i, i])
    max_row = i
    for k in range(i + 1, n_total):
        if abs(system[k, i]) > max_el:
            max_el = abs(system[k, i])
            max_row = k
    system[i], system[max_row] = system[max_row].copy(), system[i].copy()
```

```
# Eliminação
```

```
for k in range(i + 1, n_total):
    c = -system[k, i] / system[i, i]
    system[k, i:] += c * system[i, i:]
```

```

# Substituição para trás
res = np.zeros(n_total)
for i in range(n_total - 1, -1, -1):
    res[i] = (system[i, -1] - np.dot(system[i, i+1:n_total], res[i+1:n_total])) / system[i, i]

forcas_barras = res[:B]

# 4. ESTATÍSTICAS E TABELA [cite: 22, 52]
print("\n" + "="*50)
print(f'{"BARRA":<10} | {"FORÇA (N)":<15} | {"CLASSIFICAÇÃO"}')
print("-" * 50)
for i, f in enumerate(forcas_barras):
    tipo = "TRAÇÃO (+)" if f > 1e-3 else ("COMPRESSÃO (-)" if f < -1e-3 else "NULA") # [cite: 21]
    print(f'{"i":<10} | {"f":15.2f} | {"tipo}")

print("="*50)
print(f'Força Máxima: {np.max(np.abs(forcas_barras)):.2f} N')
print(f'Força Média: {np.mean(np.abs(forcas_barras)):.2f} N')

# 5. SAÍDAS GRÁFICAS OBRIGATÓRIAS
fig, axs = plt.subplots(2, 2, figsize=(14, 10))

# Gráfico 1: Treliza Original [cite: 24]
for i in range(B):
    n1, n2 = barra[i]
    axs[0,0].plot([coords_x[n1], coords_x[n2]], [coords_y[n1], coords_y[n2]], 'k-o')
for i in range(N):
    axs[0,0].text(coords_x[i], coords_y[i], f' {i}', color='blue', fontsize=12)
axs[0,0].set_title("1. Treliza Original (Nós Numerados)")

# Gráfico 2: Mapa de Calor (Tração/Compressão) [cite: 25, 26]
for i in range(B):
    n1, n2 = barra[i]
    cor = 'red' if forcas_barras[i] < -1e-3 else ('blue' if forcas_barras[i] > 1e-3 else 'gray')
    axs[0,1].plot([coords_x[n1], coords_x[n2]], [coords_y[n1], coords_y[n2]], color=cor, lw=3)
axs[0,1].set_title("2. Esforços Internos (Vermelho: Comp, Azul: Tração)")

# Gráfico 3: Magnitude das Forças [cite: 27]
axs[1,0].bar(range(B), np.abs(forcas_barras), color='skyblue')
axs[1,0].set_title("3. Magnitude das Forças por Barra")
axs[1,0].set_xlabel("Índice da Barra")

# Gráfico 4: Histograma [cite: 28]
axs[1,1].hist(forcas_barras, bins=10, color='green', alpha=0.7)
axs[1,1].set_title("4. Histograma da Distribuição de Forças")

plt.tight_layout()
plt.show()

```

```
# Executar
if __name__ == "__main__":
    solucao_trelica()
```

```
... --- SISTEMA DE ANÁLISE DE TRELIÇAS ISOSTÁTICAS ---
      Número de nós (N): 
```